SLOT 15

PERSISTENT LIST RENDERING, SYNC-UP CONCEPT, EXPORT / BACKUP JSON

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Persistent List Rendering
- Sync-Up Concept
- Export / Backup JSON

1 Persistent List Rendering

◈ Khái niệm cốt lõi sinh viên phải hiểu

Persistent List là danh sách dữ liệu được **lưu trữ lâu dài trong bộ nhớ cục bộ** của ứng dụng (local storage), thay vì chỉ tồn tại tạm thời trong RAM.

Khi ứng dụng được **đóng và mở lại**, danh sách vẫn có thể **được đọc lại từ bộ nhớ và hiển thị lại trên giao diện**.

Trong Flutter, danh sách thường được hiển thị bằng **ListView**, nhưng **nguồn dữ liệu của ListView phải đến từ storage** như:

Local database
File dữ liệu
Hoặc bộ nhớ lưu trữ ứng dụng

Nếu danh sách chỉ tồn tại trong **biến tạm của chương trình**, khi:

tắt app
reload app
crash app

→ **toàn bộ dữ liệu sẽ bị mất.**

Vì vậy, ứng dụng thực tế phải:

lưu dữ liệu
đọc dữ liệu khi app khởi động
render lại danh sách từ dữ liệu đã lưu

◈ Mục tiêu học tập

Sau khi học xong phần này, sinh viên phải có khả năng:

Hiểu sự khác nhau giữa:

danh sách **tạm thời trong bộ nhớ**
danh sách **lưu trữ bền vững**

Hiển thị danh sách dữ liệu trong Flutter bằng:

ListView.builder

Tải dữ liệu từ storage khi ứng dụng khởi động

Cập nhật danh sách khi:

thêm item
xoá item
chỉnh sửa item

Áp dụng vào các ứng dụng phổ biến:

Todo App
Note App
App quản lý chi tiêu
App danh sách sản phẩm

◈ Ý nghĩa sư phạm

Nếu danh sách **không được lưu trữ bền vững**:

người dùng **mất dữ liệu khi đóng app**

PhucPT10@fpt.edu.vn

ứng dụng trở nên **không đáng tin cậy**
trải nghiệm người dùng **rất kém**

Persistent list giúp sinh viên hiểu rằng:

Ứng dụng thật không chỉ hiển thị dữ liệu — mà phải duy trì dữ liệu lâu dài.

2 Sync-Up Concept (Data Synchronization)

◈ Khái niệm cốt lõi sinh viên phải hiểu

Data Synchronization (Sync) là quá trình:

đảm bảo dữ liệu **giữa các nguồn khác nhau luôn giống nhau hoặc được cập nhật đúng cách**

Ví dụ:

dữ liệu trong **bộ nhớ**
dữ liệu trong **local storage**
dữ liệu trên **server**

Trong ứng dụng đơn giản, sync thường xảy ra giữa:

UI
local database

Ví dụ:

User thêm một Todo:

1 UI thêm item vào list
2 dữ liệu được **lưu xuống storage**
3 danh sách trên màn hình **cập nhật lại**

Nếu không có cơ chế sync:

UI hiển thị một dữ liệu
storage lưu dữ liệu khác

→ gây **lệch dữ liệu (data inconsistency)**

◈ Mục tiêu học tập

Sau bài học này, sinh viên phải có thể:

Hiểu khái niệm **data synchronization**
Thiết kế luồng dữ liệu cơ bản:

```
User Action
     ↓
Update Storage
     ↓
Reload List
     ↓
Render UI
```

Cập nhật danh sách ngay khi dữ liệu thay đổi
Tránh lỗi phổ biến:
    UI không refresh
    dữ liệu không được lưu
    danh sách hiển thị sai
Áp dụng trong các app như:
    Todo App
    Note App
    Expense Tracker
    Shopping List

◈ Ý nghĩa sư phạm

Nếu **không có cơ chế sync**:

dữ liệu hiển thị có thể **không đúng**
người dùng **thêm dữ liệu nhưng không thấy trong danh sách**
ứng dụng trở nên **khó tin cậy**

PhucPT10@fpt.edu.vn

Sinh viên học được tư duy quan trọng:
UI chỉ là lớp hiển thị — dữ liệu thật phải được quản lý và đồng bộ đúng cách.

3 Export / Backup JSON

◈ Khái niệm cốt lõi sinh viên phải hiểu

Backup dữ liệu là quá trình:
sao lưu dữ liệu của ứng dụng
lưu vào một file bên ngoài
để có thể **khôi phục lại sau này**.
Một định dạng phổ biến để export dữ liệu là **JSON**.
JSON được sử dụng vì:
dễ đọc
nhẹ
phổ biến
dễ chuyển đổi sang object trong code
Ví dụ dữ liệu Todo:

```
[
 { "title": "Buy milk", "done": false },
 { "title": "Study Flutter", "done": true }
]
```

Ứng dụng có thể:
export danh sách Todo → file JSON
import JSON → khôi phục dữ liệu

◈ Mục tiêu học tập

Sau phần này sinh viên phải có khả năng:
Chuyển object Dart thành JSON bằng **dart:convert**
Export dữ liệu danh sách thành file JSON
Hiểu quy trình backup dữ liệu:

```
App Data
   ↓
Convert to JSON
   ↓
Save to File
```

Hiểu quy trình restore dữ liệu:

```
Load JSON File
    ↓
Decode JSON
    ↓
Convert to Object
```

Áp dụng vào các ứng dụng thực tế:
Backup danh sách Todo
Backup ghi chú
Backup lịch sử chi tiêu
Export danh sách sản phẩm

◈ Ý nghĩa sư phạm

Nếu ứng dụng **không có backup dữ liệu**:
khi người dùng **xoá app**
khi **đổi điện thoại**
khi **app gặp lỗi**
→ toàn bộ dữ liệu **có thể bị mất vĩnh viễn**.
Backup giúp sinh viên hiểu rằng:

PhucPT10@fpt.edu.vn

Dữ liệu người dùng là tài sản quan trọng của ứng dụng.

TƯ DUY CỐT LÕI SAU SLOT 15

Sau bài học này, sinh viên phải hiểu rằng:
Ứng dụng thực tế cần:

lưu dữ liệu bền vững
hiển thị dữ liệu chính xác
đồng bộ dữ liệu khi thay đổi
bảo vệ dữ liệu bằng backup

Nói cách khác:
App thật không chỉ hiển thị dữ liệu — mà còn phải duy trì và bảo vệ dữ liệu.

TÀI LIỆU THAM KHẢO

Flutter & Storage

Flutter Official Documentation
https://docs.flutter.dev

JSON trong Dart

Dart JSON Guide
https://dart.dev/guides/json

Flutter List Rendering

Flutter ListView Tutorial
https://docs.flutter.dev/cookbook/lists

JSON Serialization Guide

Flutter JSON serialization
https://docs.flutter.dev/data-and-backend/serialization/json

Video học Flutter dễ hiểu

Flutter Official YouTube
https://youtube.com/flutterdev

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 Persistent List Rendering

◈ Khái niệm quan trọng (PHẢI HIỂU ĐÚNG)

Persistent List Rendering là việc **hiển thị danh sách trên UI dựa trên dữ liệu được lưu trữ bền vững (persistent storage)** thay vì dữ liệu tạm trong RAM.

Trong Flutter, danh sách thường được hiển thị bằng widget như
ListView.builder.

Tuy nhiên, **ListView chỉ là công cụ hiển thị**.

Dữ liệu thực sự phải đến từ:
- Local database
- File dữ liệu
- Local storage

Khái niệm quan trọng:
UI phải phản ánh dữ liệu thật (source of truth).

Điều này có nghĩa:
- Widget **không nên giữ dữ liệu chính**
- Widget chỉ **đọc dữ liệu và hiển thị**

Ví dụ trong:
- Todo App
- Note App
- Expense Tracker

Danh sách phải được **load từ database khi app khởi động**.

◈ Kỹ thuật / thao tác sinh viên phải THÀNH THẠO

Sinh viên cần làm được:
- Đọc dữ liệu từ local storage
- Hiển thị danh sách bằng
  ListView
- Load dữ liệu khi app khởi động:

```
Future<void> loadData()
```

- Cập nhật danh sách khi:
  - thêm item
  - xoá item
  - chỉnh sửa item
- Dùng:
  - async / await
  - setState để refresh UI

Ví dụ luồng dữ liệu cơ bản:

```
Local Storage
      ↓
Load Data
      ↓
ListView Render
```

◈ Lỗi thường gặp của sinh viên mới

1 Render danh sách từ biến tạm

Danh sách lưu trong widget state
Không lưu vào storage

Kết quả:
reload app → mất dữ liệu

Nguyên nhân:
chưa hiểu persistent data

PhucPT10@fpt.edu.vn

2 Không await khi đọc database

Ví dụ sai:

```
var data = loadTodos();
```

UI render trước khi dữ liệu load xong.

3 UI không cập nhật khi dữ liệu thay đổi

Ví dụ:

thêm dữ liệu vào database
nhưng không reload list

Nguyên nhân:

quên gọi `setState()`

◈ Best Practices (Thực hành chuẩn)

UI luôn render từ **source of truth (database)**
Widget **không giữ dữ liệu chính**
Luồng chuẩn:

```
Database
   ↓
Load data
   ↓
UI render
```

Sau mỗi thay đổi:

```
Update DB
   ↓
Reload list
   ↓
Render UI
```

◈ CỐT LÕI vs NÂNG CAO

🟢 **Cốt lõi**

Hiển thị list từ database
Load dữ liệu khi app start
Refresh UI khi dữ liệu thay đổi

🟡 **Nâng cao**

State management (Provider / Bloc)
Lazy loading
Large list optimization

◈ Ý nghĩa sư phạm

Nếu danh sách **không render từ persistent storage**:

dữ liệu mất khi đóng app
app không đáng tin cậy

Sinh viên phải hiểu:
UI chỉ hiển thị dữ liệu — storage mới là nơi dữ liệu thật tồn tại.

2 Sync-Up Concept (Data Synchronization)

◈ Khái niệm quan trọng (PHẢI HIỂU ĐÚNG)

Data Synchronization (Sync) là quá trình **đảm bảo dữ liệu giữa các nơi khác nhau luôn giống nhau hoặc được cập nhật đúng thứ tự**.

Các nguồn dữ liệu thường gặp:

UI state
Local database

PhucPT10@fpt.edu.vn

API server

Ví dụ:

User thêm Todo:

```
User action
    ↓
Update local DB
    ↓
Update UI
```

Trong ứng dụng có server:

```
User action
    ↓
Local DB
    ↓
API server
```

Nếu không sync đúng:

UI hiển thị dữ liệu khác

server lưu dữ liệu khác

→ gọi là **data inconsistency**.

◈ Kỹ thuật / thao tác sinh viên phải THÀNH THẠO

Sinh viên cần hiểu và thực hiện được:

Luồng dữ liệu cơ bản:

```
UI
↓
Local storage
↓
Reload UI
```

Cập nhật dữ liệu theo thứ tự đúng

Ví dụ Todo App:

```
Add Todo
    ↓
Save to DB
    ↓
Reload list
```

Kiểm tra trạng thái sync.

◈ Lỗi thường gặp của sinh viên mới

1 UI update nhưng database không update

Ví dụ:

thêm item vào list

nhưng không lưu DB

App restart → mất dữ liệu.

2 Update sai thứ tự

Ví dụ:

```
Update UI
↓
Save DB
```

Nếu save DB fail → UI sai.

3 Không xử lý khi sync thất bại

Ví dụ:

lưu DB lỗi

PhucPT10@fpt.edu.vn

UI vẫn hiển thị item

◈ Best Practices

Luồng dữ liệu chuẩn:

```
User action
    ↓
Update database
    ↓
Reload data
    ↓
Render UI
```

Không nên:

```
Update UI first
```

Ngoài ra:

luôn kiểm tra kết quả save data
đảm bảo dữ liệu được ghi thành công

◈ CỐT LÕI vs NÂNG CAO

🟢 **Cốt lõi**

hiểu luồng dữ liệu UI → storage
cập nhật UI khi storage thay đổi

🟡 **Nâng cao**

conflict resolution
background sync
multi-device sync

◈ Ý nghĩa sư phạm

Nếu **không có cơ chế sync**:

dữ liệu UI và storage khác nhau
app hiển thị sai thông tin

Sinh viên học được:
dữ liệu phải được cập nhật theo một luồng logic rõ ràng.

3 Export / Backup JSON

◈ Khái niệm quan trọng (PHẢI HIỂU ĐÚNG)

Backup dữ liệu là quá trình **sao lưu dữ liệu ứng dụng ra một file bên ngoài**.
Định dạng phổ biến là
JSON.
JSON được dùng vì:

dễ đọc
nhẹ
phổ biến
dễ chuyển đổi sang object

Trong Dart, JSON được xử lý bằng thư viện:
dart:convert
Quy trình export:

```
App data
↓
Convert to JSON
↓
Save to file
```

Quy trình import:

PhucPT10@fpt.edu.vn

```
Load file
↓
Decode JSON
↓
Convert to objects
```

◈ Kỹ thuật / thao tác sinh viên phải THÀNH THẠO

Sinh viên cần làm được:

Chuyển object Dart → JSON

`jsonEncode(data)`

Chuyển JSON → object

`jsonDecode(data)`

Export danh sách Todo / Note

Import lại dữ liệu từ JSON

Ví dụ ứng dụng:

Backup Todo App

Export ghi chú

Backup Expense Tracker

◈ Lỗi thường gặp của sinh viên mới

1 Serialize object sai

Ví dụ:

object không có `toJson()`.

2 JSON structure không đúng

Ví dụ:

list object nhưng export thành string sai format.

3 Import JSON không kiểm tra dữ liệu

Nếu JSON sai format → app crash.

◈ Best Practices

Mỗi model nên có:

`toJson()`

`fromJson()`

Chuẩn hóa format JSON

Kiểm tra dữ liệu khi import

Cho phép backup định kỳ.

◈ CỐT LÕI vs NÂNG CAO

🟢 **Cốt lõi**

encode object → JSON

decode JSON → object

export / import dữ liệu

🟡 **Nâng cao**

encrypted backup

cloud backup

automatic backup

◈ Ý nghĩa sư phạm

Nếu **không có backup**:

người dùng đổi điện thoại → mất dữ liệu

PhucPT10@fpt.edu.vn

xoá app → mất toàn bộ dữ liệu

Sinh viên hiểu rằng:

dữ liệu người dùng là tài sản quan trọng của ứng dụng.

🎯 TƯ DUY CỐT LÕI SAU SLOT 15

Sau khi học xong phần này, sinh viên phải hiểu:

Một ứng dụng thật cần:

lưu dữ liệu bền vững

đồng bộ dữ liệu đúng cách

backup dữ liệu để bảo vệ người dùng

Tư duy quan trọng:

Dữ liệu trong app không chỉ để hiển thị — mà phải được lưu trữ, đồng bộ và bảo vệ.

.

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

Persistent List Rendering – Data Sync – JSON Backup

Trong các ứng dụng Flutter thực tế, dữ liệu không chỉ được hiển thị tạm thời trên giao diện.
Một ứng dụng thực tế phải đảm bảo:

dữ liệu được **lưu trữ bền vững**
giao diện **luôn phản ánh dữ liệu thật**
dữ liệu có thể **backup và khôi phục**

Ba kỹ thuật trong Slot 15 chính là nền tảng cho điều đó.

1 Persistent List Rendering trong ứng dụng thực tế

Khái niệm thực tiễn

Trong ứng dụng thật, danh sách dữ liệu luôn được **lưu trong storage**, sau đó mới được **render ra UI**.
Ví dụ:

Todo list
Notes list
Shopping list
Expense list

Flutter thường hiển thị danh sách bằng widget như:

ListView
ListView.builder
ListTile
Card

Nhưng điều quan trọng:
ListView **không phải là nơi chứa dữ liệu**, nó chỉ hiển thị dữ liệu.
Dữ liệu phải đến từ:

local storage
database
API

Ví dụ 1 – Todo App

Luồng hoạt động:

```
User thêm task
↓
Task được lưu vào local storage
↓
Danh sách được load lại
↓
UI hiển thị task mới
```

Ví dụ dữ liệu:

```
Todo List
- Buy milk
- Study Flutter
- Finish assignment
```

Quy trình render:

```
Load data from storage
↓
Convert to List<Todo>
↓
Render with ListView.builder
```

Ví dụ 2 – Notes App

Trong Notes App:

mỗi note được lưu vào database

PhucPT10@fpt.edu.vn

khi app mở → load tất cả notes

Luồng dữ liệu:

```
Open App
↓
Load Notes from database
↓
Render notes list
```

Nếu không dùng persistent storage:

đóng app → mất toàn bộ notes.

Ví dụ 3 – Shopping List

Shopping list thường gồm:

```
Milk
Bread
Eggs
Coffee
```

Danh sách này phải:

được lưu lại

hiển thị lại khi mở app.

Bài học thực tế

Sinh viên phải hiểu:

UI luôn được tạo từ dữ liệu lưu trữ, không phải từ dữ liệu tạm trong widget.

2 Sync-Up Concept giữa Data và UI

Khái niệm thực tiễn

Data Sync nghĩa là:

UI luôn phản ánh đúng trạng thái dữ liệu hiện tại.

Trong Flutter, khi dữ liệu thay đổi:

UI phải rebuild.

Điều này thường được thực hiện thông qua:

StatefulWidget

setState

Ví dụ thực tế

1 Thêm item

```
User nhấn Add
↓
Item được thêm vào list
↓
Save vào storage
↓
setState()
↓
UI hiển thị item mới
```

2 Xóa item

```
User nhấn Delete
↓
Item bị xóa khỏi list
↓
Update storage
↓
```


PhucPT10@fpt.edu.vn

```
setState()
↓
UI cập nhật
```

3 Chỉnh sửa item

```
User edit task
↓
Update database
↓
Reload data
↓
UI render lại
```

Ví dụ trong Todo App

Trước khi thêm task:

```
Todo List
- Buy milk
```

Sau khi thêm task:

```
Todo List
- Buy milk
- Study Flutter
```

Nếu không sync đúng:

UI hiển thị task mới
nhưng database không có task đó.

→ dữ liệu sai.

Bài học thực tế

Sinh viên cần hiểu:
UI chỉ là "bản phản chiếu" của dữ liệu.
Nếu dữ liệu thay đổi → UI phải cập nhật.

3 Export / Backup JSON trong ứng dụng

Khái niệm thực tiễn

Trong ứng dụng thật, dữ liệu người dùng có thể cần:

backup
chia sẻ
chuyển sang thiết bị khác

Một cách phổ biến là export dữ liệu sang:
JSON
Trong Dart, JSON được xử lý bằng thư viện:
dart:convert

Ví dụ JSON của Todo list

```
[
  {
    "title": "Buy milk",
    "completed": false
  },
  {
    "title": "Study Flutter",
    "completed": true
  }
]
```


PhucPT10@fpt.edu.vn

Use cases thực tế

1 Backup dữ liệu

User có thể export dữ liệu:

```
Todo list
Notes
Expense data
```

thành file JSON.

2 Restore dữ liệu

User cài lại app:

```
Import JSON
↓
Decode JSON
↓
Restore data
```

3 Debug ứng dụng

Developer có thể:

- export dữ liệu
- kiểm tra JSON
- tìm lỗi.

4 TÌNH HUỐNG THỰC TẾ

Tình huống 1 – Todo App

Chức năng:

- thêm task
- xóa task
- lưu task
- backup JSON

Luồng:

```
Add task
↓
Save to local storage
↓
Render list
↓
Export JSON (backup)
```

Tình huống 2 – Notes App

Chức năng:

- tạo note
- lưu note
- hiển thị danh sách note

Luồng:

```
Create note
↓
Save note to database
↓
Load notes
↓
Render notes list
```

Nếu không persistent storage:

PhucPT10@fpt.edu.vn

đóng app → mất note.

Tình huống 3 – Habit Tracker

Habit tracker lưu dữ liệu:

```
Drink Water
Exercise
Read Book
```

Danh sách phải:

- hiển thị mỗi ngày
- được lưu lại
- có thể export JSON.

5 MINI CASE STUDY – Todo App với Persistence

Mô hình dữ liệu

```
Todo
- id
- title
- completed
```

Bước 1 – Render list

UI hiển thị bằng:

- **ListView.builder**

Danh sách được load từ database.

Bước 2 – Sync State

Khi thêm task:

```
Add Todo
↓
Save DB
↓
setState()
↓
UI rebuild
```

Bước 3 – Export JSON

Dữ liệu Todo được chuyển sang JSON:

```
List<Todo>
↓
map()
↓
toJson()
↓
jsonEncode()
```

User có thể:

- backup file
- import lại dữ liệu.

6 TƯ DUY ỨNG DỤNG THỰC TẾ

Sau Slot 15, sinh viên cần hiểu một tư duy quan trọng trong phát triển ứng dụng:

1 UI không phải là dữ liệu

UI chỉ hiển thị dữ liệu.

PhucPT10@fpt.edu.vn

2 Dữ liệu phải được lưu trữ

App thật luôn cần:

- persistence
- data integrity

3 Dữ liệu người dùng phải được bảo vệ

Backup là cơ chế quan trọng để:

- tránh mất dữ liệu
- chuyển dữ liệu sang thiết bị khác.

🎯 KẾT LUẬN

Ba kỹ thuật trong Slot 15:

- Persistent List Rendering
- Data Synchronization
- JSON Backup

là **nền tảng của mọi ứng dụng có dữ liệu người dùng** như:

- Todo App
- Notes App
- Expense Tracker
- Habit Tracker
- Shopping List

Nếu không có các cơ chế này:

- dữ liệu sẽ bị mất
- UI hiển thị sai dữ liệu
- người dùng không thể backup thông tin.

✅ Sau phần này sinh viên phải hiểu:

Một ứng dụng thật không chỉ hiển thị dữ liệu —
mà phải **lưu trữ, đồng bộ và bảo vệ dữ liệu của người dùng.**

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

GIẢI THÍCH CHUYÊN SÂU & NỀN TẢNG HỌC THUẬT

Persistent List Rendering – Data Sync – JSON Backup

Trong các ứng dụng Flutter hiện đại, UI không phải là một giao diện tĩnh.
Flutter hoạt động theo triết lý **UI được tạo ra từ dữ liệu và trạng thái ứng dụng**.
Do đó, ba khái niệm quan trọng trong Slot 15 là:

- Persistent List Rendering
- Data Synchronization
- JSON Data Representation

Ba yếu tố này tạo thành **nền tảng quản lý dữ liệu trong ứng dụng Flutter**.

PHẦN A — PERSISTENT LIST RENDERING

1 Bản chất của dữ liệu danh sách trong Flutter

Trong ứng dụng Flutter, danh sách dữ liệu thường được biểu diễn dưới dạng:

```
List<T>
```

Ví dụ:

```
List<Todo>
List<Note>
List<Product>
```

Danh sách này có thể được lưu trữ trong:

- bộ nhớ tạm (RAM)
- local database
- file JSON
- API server

Tuy nhiên, trong ứng dụng thực tế, dữ liệu quan trọng phải được lưu trong **persistent storage**.
Điều này giúp đảm bảo:

- dữ liệu không mất khi app restart
- dữ liệu có thể được khôi phục
- dữ liệu có thể được đồng bộ.

2 UI trong Flutter được tạo ra từ dữ liệu

Flutter sử dụng mô hình **data-driven UI**.
Điều này có nghĩa:

```
UI = function(state)
```

Trong đó:

- **state** là dữ liệu hiện tại của ứng dụng
- **UI** là kết quả render từ state

Ví dụ:

```
State (Todo List)
↓
["Buy milk", "Study Flutter"]
↓
UI
↓
ListView hiển thị 2 item
```

Nếu state thay đổi:

```
State
↓
["Buy milk", "Study Flutter", "Exercise"]
```

UI sẽ tự động được rebuild.

3 Persistent List Rendering

Khái niệm này có nghĩa:

PhucPT10@fpt.edu.vn

Danh sách UI được render từ dữ liệu lưu trữ bền vững.
Ví dụ trong Todo App:

```
Local Storage
↓
Load Todos
↓
List<Todo>
↓
Render UI
```

Widget thường dùng để render danh sách là:

ListView
ListView.builder

Trong đó:

ListView.builder giúp render danh sách lớn hiệu quả.

4 Source of Truth (Khái niệm quan trọng)

Trong kiến trúc ứng dụng, dữ liệu phải có **nguồn dữ liệu chính (source of truth)**.
Ví dụ:

```
Database = Source of Truth
```

UI chỉ là:

```
representation of data
```

Nếu UI giữ dữ liệu riêng → dễ gây lỗi.
Do đó:

```
Database
↓
Load data
↓
Render UI
```

PHẦN B — SYNC-UP CONCEPT

1 Reactive UI trong Flutter

Flutter sử dụng mô hình **reactive UI**.
Reactive UI có nghĩa:
UI tự động thay đổi khi dữ liệu thay đổi.
Cơ chế này được thực hiện thông qua:

StatefulWidget
setState

2 Chu trình cập nhật UI trong Flutter

Khi dữ liệu thay đổi, Flutter thực hiện chu trình sau:

```
State change
↓
setState()
↓
Widget rebuild
↓
UI update
```

Giải thích từng bước:

State change

Dữ liệu trong ứng dụng thay đổi.
Ví dụ:

```
todos.add(newTodo)
```


PhucPT10@fpt.edu.vn

setState()
setState thông báo cho Flutter rằng:
trạng thái đã thay đổi.

Widget rebuild
Flutter sẽ:
    gọi lại hàm `build()`
    tạo lại widget tree.

UI update
UI mới được render trên màn hình.

3 Rebuild Mechanism của Flutter
Flutter không sửa từng phần UI.
Thay vào đó:
    Flutter **tạo lại widget tree mới**
    so sánh với tree cũ.
Cơ chế này gọi là:
`Widget rebuild`
Nhờ vậy Flutter có thể:
    cập nhật UI rất nhanh
    giữ code đơn giản.

4 Sync giữa Data và UI
Mục tiêu của sync là:
`Data state = UI state`
Nếu dữ liệu thay đổi nhưng UI không rebuild:
`Data ≠ UI`
Điều này dẫn đến:
    UI hiển thị sai dữ liệu
    user experience kém.

PHẦN C — JSON DATA MODEL
1 JSON là gì?
JSON là một định dạng dữ liệu phổ biến dùng để:
    lưu trữ dữ liệu
    trao đổi dữ liệu giữa hệ thống.
Ví dụ JSON:
```
{
 "tasks": [
   { "title": "Buy milk", "done": false }
 ]
}
```

2 JSON trong Flutter
Flutter sử dụng thư viện:
    dart:convert
Để chuyển đổi giữa:
`Dart Object ↔ JSON`
Ví dụ:
```
Todo Object
↓
toJson()
```


PhucPT10@fpt.edu.vn

```
↓
Map
↓
jsonEncode()
↓
JSON String
```

3 Serialization và Deserialization

Hai khái niệm quan trọng:

Serialization

Chuyển object → JSON.

```
Todo
↓
Map
↓
JSON
```

Deserialization

Chuyển JSON → object.

```
JSON
↓
Map
↓
Todo object
```

4 Vì sao JSON được dùng phổ biến?

JSON có nhiều ưu điểm:

- dễ đọc
- nhẹ
- phổ biến
- tương thích với API.

Do đó JSON thường được dùng cho:

- API communication
- data export
- backup file.

PHẦN D — DATA FLOW TRONG FLUTTER APP

Trong một ứng dụng Flutter, dữ liệu thường di chuyển theo một luồng rõ ràng.

Ví dụ Todo App:

```
User Action
↓
Update List
↓
setState()
↓
Widget rebuild
↓
ListView render
```

Giải thích:

User Action

User nhấn:

- Add
- Delete

PhucPT10@fpt.edu.vn

Edit.

Update List
Danh sách dữ liệu thay đổi.
```
todos.add(task)
```

setState()
Thông báo cho Flutter rằng dữ liệu đã thay đổi.

Widget rebuild
Flutter gọi lại:
```
build()
```

ListView render
Danh sách mới được hiển thị bằng:
ListView.builder

PHẦN E — KẾT NỐI VỚI CÁC BÀI ĐÃ HỌC TRƯỚC
Kiến thức trong Slot 15 không đứng riêng lẻ.
Nó liên kết trực tiếp với các chủ đề trước đó trong Flutter.

1 Local Database
Sinh viên đã học:
lưu dữ liệu local
đọc dữ liệu từ database.
Slot 15 mở rộng bằng cách:
```
Load DB
↓
Render List
```

2 API Sync
Trong ứng dụng thật:
```
Server API
↓
Local storage
↓
UI
```
Điều này giúp:
offline usage
faster UI.

3 State Management
Flutter có nhiều cách quản lý state:
setState
Provider
Bloc
Riverpod
Trong Slot 15, sinh viên học **tư duy cơ bản của state synchronization**.

🎯 KẾT LUẬN HỌC THUẬT
Ba khái niệm trong Slot 15 thể hiện tư duy cốt lõi của ứng dụng Flutter hiện đại:

PhucPT10@fpt.edu.vn

1 Data-driven UI

```
UI = function(state)
```

2 Reactive Architecture

```
State change
↓
Widget rebuild
↓
UI update
```

3 Structured Data Model

```
Object
↓
JSON
↓
Storage / API
```

🎓 TƯ DUY QUAN TRỌNG SINH VIÊN PHẢI NẮM

Sau khi học phần này, sinh viên phải hiểu:

1. UI chỉ là biểu diễn của dữ liệu.
2. Dữ liệu thay đổi → UI phải cập nhật.
3. JSON là cách phổ biến để lưu và trao đổi dữ liệu.
4. Ứng dụng thật phải có:
   persistent data
   synchronization
   backup.

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1 Persistent List Rendering

◈ Kinh nghiệm thực tế

Trong ứng dụng Flutter thật, danh sách dữ liệu thường có các đặc điểm:

dữ liệu **thay đổi liên tục**
số lượng item **có thể rất lớn**
mỗi item có **UI phức tạp**

Ví dụ:
Todo App:

```
Todo List
- Buy milk
- Study Flutter
- Exercise
- Finish project
```

Danh sách này thường được hiển thị bằng:

ListView
ListView.builder

Trong thực tế:

danh sách có thể chứa **hàng trăm hoặc hàng nghìn item**
UI cần render **hiệu quả và nhanh**

Do đó việc thiết kế list UI đúng cách là rất quan trọng.

⚠ Lỗi phổ biến

✕ 1. Rebuild toàn bộ UI không cần thiết

Sinh viên thường viết:

```
Column
  children:
    List.generate(...)
```

Điều này làm:

toàn bộ list được build lại
hiệu năng giảm.

Trong khi **ListView.builder** chỉ render item khi cần.

✕ 2. Không dùng builder pattern

Một số sinh viên viết:

```
ListView(
 children: [...]
)
```

Nếu list lớn:

tất cả item được build cùng lúc
gây lag UI.

✕ 3. Item widget quá phức tạp

Ví dụ:

item chứa nhiều logic
nhiều nested widgets
nhiều setState bên trong.

Điều này làm:

khó maintain code
rebuild chậm.

✕ 4. Render list từ biến tạm

Ví dụ:

PhucPT10@fpt.edu.vn

```
List<String> todos = [];
```
Nếu danh sách này không lưu vào storage:
restart app → mất dữ liệu.

Tips thực tế

Tip 1 – Tách item widget

Thay vì viết:
```
ListView.builder(
 itemBuilder: (...)
)
```
Trong đó item rất dài.
Hãy tách thành:
```
TodoItemWidget
```
Ví dụ:
```
TodoItemWidget(todo: todos[index])
```
Điều này giúp:
code rõ ràng
dễ maintain.

Tip 2 – Dùng builder pattern

Luôn ưu tiên:
ListView.builder
Vì nó:
render item khi cần
tiết kiệm bộ nhớ.

Tip 3 – Tránh logic nặng trong build()

Hàm build nên:
chỉ tạo UI
không chứa logic phức tạp.

Best Practices

Các ứng dụng Flutter production thường:
✓ mỗi item là **widget riêng**
```
TodoItemWidget
NoteItemWidget
ProductItemWidget
```
✓ UI chỉ render dữ liệu
✓ logic nằm trong:
controller
service
state management.

2 Sync-Up Concept

Kinh nghiệm thực tế

Một trong những vấn đề phổ biến nhất trong ứng dụng Flutter là:
UI không phản ánh đúng dữ liệu.
Điều này xảy ra khi state thay đổi nhưng UI không rebuild.
Flutter sử dụng:
StatefulWidget
setState
để đồng bộ UI và state.

PhucPT10@fpt.edu.vn

⚠ Lỗi phổ biến

✕ 1. Thay đổi list nhưng quên setState

Ví dụ:

```
todos.add(task);
```

Nếu không gọi:

```
setState()
```

UI sẽ không update.

✕ 2. UI update trước khi dữ liệu được lưu

Ví dụ:

```
setState()
saveToDatabase()
```

Nếu database save fail:

UI hiển thị sai dữ liệu.

✕ 3. Multiple sources of truth

Sinh viên đôi khi lưu dữ liệu ở nhiều nơi:

```
List in UI
List in DB
List in memory
```

Điều này dễ gây:

data mismatch.

✕ 4. Không reload dữ liệu sau khi update database

Ví dụ:

```
saveTodo()
```

Nhưng không load lại list.

💡 Tips thực tế

Tip 1 – Luồng dữ liệu rõ ràng

Luôn theo luồng:

```
User action
↓
Update data
↓
setState
↓
UI rebuild
```

Tip 2 – Kiểm tra state trước khi update UI

Ví dụ:

```
await saveTodo();
setState(...)
```

Tip 3 – Giữ state đơn giản

State nên chỉ chứa:

```
List<Todo>
```

Không nên chứa:

logic phức tạp
nhiều biến phụ.

PhucPT10@fpt.edu.vn

🛡 Best Practices

Các ứng dụng production thường áp dụng:

Single Source of Truth

Chỉ có **một nguồn dữ liệu chính**.

Ví dụ:

```
Database
```

UI chỉ:

```
render data
```

Data → UI Flow

Luồng chuẩn:

```
Database
↓
Load data
↓
State
↓
UI
```

3 Export / Backup JSON

◈ Kinh nghiệm thực tế

Trong ứng dụng thực tế, backup dữ liệu rất quan trọng.

Ví dụ:

- Todo App
- Notes App
- Expense Tracker

User có thể muốn:

- chuyển dữ liệu sang điện thoại mới
- lưu trữ dữ liệu
- export dữ liệu.

Định dạng phổ biến nhất là:

- **JSON**

Flutter hỗ trợ JSON thông qua thư viện:

- **dart:convert**

⚠ Lỗi phổ biến

✕ 1. JSON structure sai

Ví dụ:

```
{
 title: Buy milk
}
```

Sai vì:

- thiếu dấu ngoặc.

✕ 2. Không xử lý dữ liệu null

Ví dụ:

```
title: null
```

Nếu code không xử lý null:

- app có thể crash.

✕ 3. Không có model class

Sinh viên đôi khi làm:

PhucPT10@fpt.edu.vn

```
Map<String, dynamic>
```
khắp nơi trong code.
Điều này làm code:
- khó đọc
- khó maintain.

❌ 4. Import JSON mà không validate dữ liệu
Nếu JSON sai:
- app crash.

💡 Tips thực tế
Tip 1 – Luôn tạo model class
Ví dụ:
```
class Todo
```
Model giúp:
- code rõ ràng
- dễ serialize.

Tip 2 – Sử dụng toJson() / fromJson()
Ví dụ:
```
toJson()
fromJson()
```
Điều này giúp:
- convert object ↔ JSON.

Tip 3 – Kiểm tra dữ liệu khi import
Trước khi decode JSON:
```
validate structure
```

🛡 Best Practices
Chuẩn hóa Data Model
Ví dụ Todo:
```
Todo
 id
 title
 completed
 createdAt
```

JSON Structure rõ ràng
Ví dụ:
```
{
 "todos": [...]
}
```

Backup định kỳ
Một số ứng dụng cho phép:
- auto backup
- export data.

🎓 BÀI HỌC QUAN TRỌNG CHO SINH VIÊN
Sau Slot 15, sinh viên phải ghi nhớ 3 nguyên tắc quan trọng trong Flutter data-driven applications:

PhucPT10@fpt.edu.vn

1 UI Flutter luôn phụ thuộc state

Nguyên tắc:

```
UI = function(state)
```

Nếu state thay đổi → UI phải rebuild.

2 List UI được render từ data model

Danh sách UI nên được tạo từ:

```
List<Model>
```

Không nên dùng:

```
temporary data
```

3 JSON giúp lưu và chia sẻ dữ liệu

JSON được dùng để:

- backup
- export
- API communication.

🎯 TƯ DUY QUAN TRỌNG TRONG ỨNG DỤNG THỰC

Một ứng dụng Flutter thực tế cần đảm bảo:

✓ dữ liệu được lưu trữ bền vững

✓ UI luôn đồng bộ với dữ liệu

✓ dữ liệu có thể backup và khôi phục

Ba kỹ thuật của Slot 15 chính là nền tảng cho điều đó.

PhucPT10@fpt.edu.vn

EXERCISE

BÀI TẬP TỔNG HỢP

) Tổng quan bài học (dành cho SV mới)
Bạn sẽ học được gì?

Danh sách Todo được vẽ ra từ List<Todo> (dữ liệu là nguồn gốc)
setState() là "nút Sync" giúp **UI cập nhật** sau khi dữ liệu thay đổi
toJson / fromJson để chuẩn hóa dữ liệu
jsonEncode để xuất dữ liệu thành **chuỗi JSON** (backup)

1) TÓM TẮT DỰ ÁN
📌 Tên dự án
Smart Todo Backup App
🎯 Mục tiêu
Xây dựng app Todo đơn giản có khả năng:

Hiển thị danh sách công việc
UI cập nhật khi dữ liệu thay đổi (thêm / toggle done)
Xuất Todo thành **JSON** để backup

Kết quả mong đợi sau bài
Sinh viên hiểu:

UI = function(state) (UI render từ dữ liệu)
state thay đổi → setState → UI rebuild
JSON dùng để backup/trao đổi dữ liệu

2) KIẾN THỨC SLOT 15 CỦNG CỐ

Kiến thức	Áp dụng trong project
Persistent List Rendering	ListView.builder render todos
Sync-Up Concept	setState() giúp UI update
JSON Data Model	Todo.toJson / Todo.fromJson
Export / Backup JSON	jsonEncode danh sách Todo

3) CHỨC NĂNG ỨNG DỤNG

Hiển thị danh sách Todo
Dùng `ListView.builder`
Thêm Todo
Nhập text → bấm "+" → thêm vào list
Đánh dấu hoàn thành
Tap vào item → toggle done → UI gạch ngang
Export dữ liệu JSON
Bấm icon Save → in JSON ra Console (bước cơ bản dễ kiểm chứng)

Ghi chú: Ở mức mới học, backup ra "console" là cách đơn giản nhất để chứng minh kỹ thuật Export JSON đã chạy đúng. Sau này có thể mở rộng lưu file thật.

PhucPT10@fpt.edu.vn

4) CÔNG NGHỆ SỬ DỤNG

Công nghệ	Mục đích
Flutter SDK	UI framework
Dart	Ngôn ngữ
VS Code	IDE
StatefulWidget	Quản lý state cho Todo list
JSON + dart:convert	Encode/Decode JSON

Thư viện dùng:

`dart:convert`

5) CẤU TRÚC THƯ MỤC DỰ ÁN

Text
```
lib
 ├── main.dart
 ├── models
 │    └── todo.dart
 ├── screens
 │    └── home_screen.dart
 ├── widgets
 │    └── todo_item.dart
 └── services
      └── json_service.dart
```

6) TÓM TẮT QUY TRÌNH THỰC HIỆN (Checklist)

Create Flutter project
Tạo model `Todo`
Tạo `HomeScreen` (StatefulWidget)
Render list bằng `ListView.builder` (**Persistent List Rendering**)
Thêm Todo mới
Toggle trạng thái done
Sync UI bằng `setState` (**Sync-Up Concept**)
Convert Todo → JSON (toJson)
Export JSON backup (jsonEncode)

7) HƯỚNG DẪN THỰC HÀNH CHI TIẾT TỪNG BƯỚC (từng thao tác)

BƯỚC 1 — TẠO PROJECT

1. Mở Terminal (trong VS Code)

VS Code → **Terminal** → **New Terminal**

2. Gõ lệnh tạo project

bash
```bash
flutter create todo_backup_app
```

3. Mở project

VS Code → **File** → **Open Folder...** → chọn `todo_backup_app`

4. Chạy thử

bash
```bash
flutter run
```

✅ **Kiểm chứng**

App chạy được trên emulator / device thật, không lỗi.

PhucPT10@fpt.edu.vn

BƯỚC 2 — TẠO TODO DATA MODEL (JSON MODEL)

1. Tạo file

Chuột phải thư mục `lib` → New Folder: `models`

Chuột phải `lib/models` → New File: `todo.dart`

2. Dán code

lib/models/todo.dartv1

```
class Todo {
  String title;
  bool done;

  Todo({required this.title, this.done = false});

  Map<String, dynamic> toJson() {
    return {
      'title': title,
      'done': done,
    };
  }

  factory Todo.fromJson(Map<String, dynamic> json) {
    return Todo(
      title: json['title'],
      done: json['done'],
    );
  }
}
```

✅ **Kiểm chứng kỹ thuật**

Không kiểm chứng bằng UI ở bước này, nhưng bạn hiểu:

`toJson()` tạo Map chuẩn để encode JSON

`fromJson()` giúp restore dữ liệu (chuẩn bị cho bài nâng cao sau này)

BƯỚC 3 — TẠO HOME SCREEN (UI + STATE)

1. Tạo file

`lib` → New Folder: `screens`

`lib/screens` → New File: `home_screen.dart`

2. Dán code (phiên bản cơ bản có Add + Toggle + ListView)

lib/screens/home_screen.dartv1

```
import 'package:flutter/material.dart';
import '../models/todo.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
```


PhucPT10@fpt.edu.vn

```
List<Todo> todos = [];
final TextEditingController controller = TextEditingController();

void addTodo() {
  if (controller.text.isEmpty) return;

  setState(() {
    todos.add(Todo(title: controller.text));
  });

  controller.clear();
}

void toggleTodo(int index) {
  setState(() {
    todos[index].done = !todos[index].done;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text("Todo Backup App")),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.all(8),
          child: Row(
            children: [
              Expanded(
                child: TextField(
                  controller: controller,
                  decoration: const InputDecoration(
                    hintText: "Enter task",
                  ),
                ),
              ),
              IconButton(
                icon: const Icon(Icons.add),
                onPressed: addTodo,
              )
            ],
          ),
        ),
        Expanded(
          child: ListView.builder(
            itemCount: todos.length,
            itemBuilder: (context, index) {
              final todo = todos[index];

              return ListTile(
                title: Text(
                  todo.title,
```


PhucPT10@fpt.edu.vn

```
                    style: TextStyle(
                      decoration:
                          todo.done ? TextDecoration.lineThrough : null,
                    ),
                  ),
                  onTap: () => toggleTodo(index),
                );
              },
            ),
          )
        ],
      ),
    );
  }
}
```

✅ **Kiểm chứng (2 kỹ thuật quan trọng)**

Persistent List Rendering

Thêm 5–10 Todo → danh sách tự render theo `todos.length`

Scroll được (nhờ `Expanded` bọc ListView)

Sync-Up Concept

Tap vào item → nó gạch ngang ngay (UI update) vì có `setState`

BƯỚC 4 — KẾT NỐI MAIN

1. Mở `lib/main.dart`

2. Thay nội dung bằng

lib/main.dartv1

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: HomeScreen(),
    );
  }
}
```

✅ **Kiểm chứng**

App mở ra là HomeScreen.

PhucPT10@fpt.edu.vn

8) BƯỚC 5 — EXPORT / BACKUP JSON (Service riêng)

8.1 Tạo JsonService

1. Tạo file

`lib` → New Folder: `services`

`lib/services` → New File: `json_service.dart`

2. Dán code

lib/services/json_service.dartv1

```
import 'dart:convert';
import '../models/todo.dart';

class JsonService {
  static String exportTodos(List<Todo> todos) {
    final list = todos.map((e) => e.toJson()).toList();
    return jsonEncode(list);
  }
}
```

✅ **Kiểm chứng**

Chưa thấy gì trên UI, nhưng service đã sẵn sàng.

8.2 Thêm nút Backup trên AppBar

1. Mở `lib/screens/home_screen.dart`

2. Import JsonService

Thêm dòng:

Dart

```
import '../services/json_service.dart';
```

3. Cập nhật AppBar có actions save

Thay AppBar thành:

lib/screens/home_screen.dartv2

```
appBar: AppBar(
  title: const Text("Todo Backup App"),
  actions: [
    IconButton(
      icon: const Icon(Icons.save),
      onPressed: () {
        final jsonData = JsonService.exportTodos(todos);
        print(jsonData);
      },
    )
  ],
),
```

✅ **Kiểm chứng Export JSON**

Test dữ liệu:

Add: `Buy milk`

Add: `Study Flutter`

Tap `Study Flutter` để done = true

PhucPT10@fpt.edu.vn

Nhấn icon Save → Console in ra JSON dạng:

```JSON
[{"title":"Buy milk","done":false},{"title":"Study Flutter","done":true}]
```

9) NÂNG CẤP CHO DỄ HIỂU: TÁCH WIDGET TODO ITEM (maintain tốt hơn)

BƯỚC 6 — Tạo widget TodoItemWidget

1. Tạo file

`lib` → folder `widgets`

`lib/widgets` → New File: `todo_item.dart`

2. Code widget

lib/widgets/todo_item.dartv1

```Dart
import 'package:flutter/material.dart';
import '../models/todo.dart';

class TodoItem extends StatelessWidget {
  final Todo todo;
  final VoidCallback onTap;

  const TodoItem({
    super.key,
    required this.todo,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(
        todo.title,
        style: TextStyle(
          decoration: todo.done ? TextDecoration.lineThrough : null,
        ),
      ),
      onTap: onTap,
    );
  }
}
```

BƯỚC 7 — Dùng TodoItem trong ListView.builder

Trong `home_screen.dart`:

Import widget:

```Dart
import '../widgets/todo_item.dart';
```

Thay `return ListTile(...)` bằng:

```Dart
```


PhucPT10@fpt.edu.vn

```
return TodoItem(
  todo: todo,
  onTap: () => toggleTodo(index),
);
```

✅ **Kiểm chứng**

UI y như cũ, nhưng code dễ đọc hơn, đúng "tách UI – logic".

10) DATA FLOW TRONG APP (Giải thích đúng trọng tâm)

```Text
User nhập TextField
↓
Nhấn "+"
↓
addTodo() thêm vào List<Todo>
↓
setState()
↓
Flutter gọi build() lại
↓
ListView.builder render theo todos
↓
Nhấn Save → exportTodos()
↓
jsonEncode → in JSON ra console
```

11) LỖI PHỔ BIẾN (và cách tự sửa)

Quên setState()

Hiện tượng: dữ liệu có đổi nhưng UI không đổi

Sửa: mọi thay đổi `todos.add`, `todos[index].done = ...` phải nằm trong `setState(() { ... })`

ListView không scroll / báo lỗi layout

Nguyên nhân: ListView nằm trong Column nhưng không được bọc `Expanded`

Sửa: dùng `Expanded(child: ListView.builder(...))`

JSON export sai

Nguyên nhân: model thiếu `toJson()` hoặc key sai

Sửa: kiểm tra `toJson()` trả đúng `'title'`, `'done'`

PhucPT10@fpt.edu.vn

12) TỔNG HỢP CODE FULL (BẢN HOÀN CHỈNH)

lib/main.dartv2

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: HomeScreen(),
    );
  }
}
```

lib/models/todo.dartv2

```
class Todo {
  String title;
  bool done;

  Todo({required this.title, this.done = false});

  Map<String, dynamic> toJson() {
    return {
      'title': title,
      'done': done,
    };
  }

  factory Todo.fromJson(Map<String, dynamic> json) {
    return Todo(
      title: json['title'],
      done: json['done'],
    );
  }
}
```

lib/services/json_service.dartv2

```
import 'dart:convert';
import '../models/todo.dart';

class JsonService {
  static String exportTodos(List<Todo> todos) {
    final list = todos.map((e) => e.toJson()).toList();
```


PhucPT10@fpt.edu.vn

```
    return jsonEncode(list);
  }
}
```

lib/widgets/todo_item.dartv2

```
import 'package:flutter/material.dart';
import '../models/todo.dart';

class TodoItem extends StatelessWidget {
  final Todo todo;
  final VoidCallback onTap;

  const TodoItem({
    super.key,
    required this.todo,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(
        todo.title,
        style: TextStyle(
          decoration: todo.done ? TextDecoration.lineThrough : null,
        ),
      ),
      onTap: onTap,
    );
  }
}
```

lib/screens/home_screen.dartv3

```
import 'package:flutter/material.dart';
import '../models/todo.dart';
import '../services/json_service.dart';
import '../widgets/todo_item.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  List<Todo> todos = [];
  final TextEditingController controller = TextEditingController();

  void addTodo() {
```


PhucPT10@fpt.edu.vn

```
    if (controller.text.isEmpty) return;

    setState(() {
      todos.add(Todo(title: controller.text));
    });

    controller.clear();
  }

  void toggleTodo(int index) {
    setState(() {
      todos[index].done = !todos[index].done;
    });
  }

  void exportJson() {
    final jsonData = JsonService.exportTodos(todos);
    print(jsonData);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Todo Backup App"),
        actions: [
          IconButton(
            icon: const Icon(Icons.save),
            onPressed: exportJson,
          )
        ],
      ),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(8),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: controller,
                    decoration: const InputDecoration(
                      hintText: "Enter task",
                    ),
                  ),
                ),
                IconButton(
                  icon: const Icon(Icons.add),
                  onPressed: addTodo,
                )
              ],
            ),
          ),
```


PhucPT10@fpt.edu.vn

```
          Expanded(
            child: ListView.builder(
              itemCount: todos.length,
              itemBuilder: (context, index) {
                final todo = todos[index];
                return TodoItem(
                  todo: todo,
                  onTap: () => toggleTodo(index),
                );
              },
            ),
          )
        ],
      ),
    );
  }
}
```

13) “Phiên bản code theo diễn biến” (SV yếu học theo từng mốc)

Version A (Render list cơ bản)

- Có: `HomeScreen`, `ListView.builder`, `todos=[]`
- Chưa có: add/toggle/export

Version B (Add Todo + setState)

- Thêm `TextField`, `IconButton +`, `addTodo()`
- Kiểm chứng: bấm + thấy item xuất hiện

Version C (Toggle done + Sync-Up)

- Thêm `toggleTodo()`, `onTap`
- Kiểm chứng: tap item → gạch ngang ngay

Version D (Export JSON)

- Thêm `Todo.toJson`, `JsonService.exportTodos`, icon Save
- Kiểm chứng: nhấn Save → console có JSON đúng

Version E (Tách widget TodoItem)

- Thêm `widgets/todo_item.dart`
- Kiểm chứng: UI không đổi, code dễ đọc hơn

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

Thuật ngữ	Giải thích đơn giản	Vai trò trong Flutter App	Ví dụ minh họa
Persistent Data	Dữ liệu được lưu trữ bền vững và không bị mất khi ứng dụng đóng	Giúp dữ liệu người dùng tồn tại lâu dài	Todo list vẫn còn sau khi mở lại app
Persistent Storage	Nơi lưu dữ liệu lâu dài như database, file hoặc local storage	Là nơi lưu dữ liệu chính của ứng dụng	Local database lưu danh sách tasks
Persistent List Rendering	Hiển thị danh sách dữ liệu từ nguồn lưu trữ bền vững	Đảm bảo UI phản ánh dữ liệu thật	Load tasks từ database rồi render list
Data Persistence	Quá trình lưu dữ liệu vào storage để sử dụng lâu dài	Giúp ứng dụng giữ dữ liệu khi restart	Lưu note vào database
Source of Truth	Nguồn dữ liệu chính và đáng tin cậy nhất của hệ thống	Tránh dữ liệu bị lệch giữa UI và storage	Database là source of truth
State	Trạng thái dữ liệu hiện tại của ứng dụng	Quyết định UI sẽ hiển thị như thế nào	List<Todo> là state của Todo screen
Stateful Widget	Widget có thể thay đổi trạng thái trong quá trình chạy	Cho phép UI cập nhật khi dữ liệu thay đổi	**StatefulWidget**
setState()	Hàm thông báo cho Flutter rằng state đã thay đổi	Trigger rebuild UI	**setState**
Widget Rebuild	Quá trình Flutter tạo lại widget tree khi state thay đổi	Cập nhật UI khi dữ liệu thay đổi	List hiển thị item mới sau khi add task
Reactive UI	Mô hình UI tự động cập nhật khi dữ liệu thay đổi	Là nguyên lý hoạt động của Flutter UI	State change → UI update
Data Synchronization (Data Sync)	Quá trình đảm bảo dữ liệu giữa các thành phần luôn giống nhau	Đồng bộ dữ liệu giữa UI và storage	Save task → UI hiển thị ngay
Data Flow	Luồng di chuyển của dữ liệu trong ứng dụng	Giúp quản lý logic ứng dụng rõ ràng	User action → update state → UI
List Data Model	Cấu trúc dữ liệu dạng danh sách trong code	Dùng để lưu nhiều item cùng loại	List<Todo>
Lazy Rendering	Chỉ render các item khi cần thiết	Tăng hiệu năng cho danh sách dài	**ListView.builder**
ListView	Widget hiển thị danh sách cuộn trong Flutter	Dùng để hiển thị nhiều item	**ListView**
List Item Widget	Widget đại diện cho một item trong danh sách	Giúp code dễ quản lý	TodoItemWidget
JSON	Định dạng dữ liệu dạng text dùng để lưu và trao đổi dữ liệu	Dùng để backup hoặc giao tiếp API	**JSON**

PhucPT10@fpt.edu.vn

JSON Serialization	Chuyển object thành JSON	Chuẩn bị dữ liệu để lưu hoặc gửi API	Todo → JSON
JSON Deserialization	Chuyển JSON thành object trong code	Tạo lại dữ liệu từ file hoặc API	JSON → Todo object
Data Model	Cấu trúc mô tả dữ liệu trong ứng dụng	Giúp tổ chức dữ liệu rõ ràng	class Todo
toJson()	Hàm chuyển object thành JSON	Dùng khi export hoặc gửi API	todo.toJson()
fromJson()	Hàm tạo object từ JSON	Dùng khi đọc dữ liệu từ JSON	Todo.fromJson(json)
Backup	Quá trình sao lưu dữ liệu	Tránh mất dữ liệu người dùng	Export todo list
Restore Data	Khôi phục dữ liệu từ file backup	Giúp user lấy lại dữ liệu	Import JSON file
Data Integrity	Tính chính xác và nhất quán của dữ liệu	Tránh dữ liệu sai hoặc mất	Sync đúng giữa UI và DB
Single Source of Truth	Chỉ có một nơi chứa dữ liệu chính	Tránh xung đột dữ liệu	Database là nguồn dữ liệu duy nhất
Local Storage	Nơi lưu dữ liệu trong thiết bị người dùng	Giúp ứng dụng hoạt động offline	SharedPreferences / SQLite

🎓 CÁCH SINH VIÊN NÊN SỬ DỤNG BẢNG THUẬT NGỮ

Sinh viên nên dùng bảng này để:

1 Hiểu thuật ngữ trong code

Ví dụ khi đọc code:

```
setState(() {
  todos.add(task);
});
```

Sinh viên phải hiểu:

State
setState
UI rebuild

2 Hiểu kiến trúc ứng dụng

Ví dụ Todo App:

```
User Action
↓
Update Data
↓
State Change
↓
setState()
↓
UI Rebuild
```

3 Hiểu cách dữ liệu được lưu trữ

```
Todo Object
↓
toJson()
↓
```


PhucPT10@fpt.edu.vn

```
JSON
↓
Save File
```

🎯 TƯ DUY QUAN TRỌNG SINH VIÊN PHẢI NẮM

Sau khi học Slot 15, sinh viên cần hiểu:

1️⃣ **UI Flutter được tạo từ state**

```
UI = function(state)
```

2️⃣ **Danh sách UI phải được render từ dữ liệu thật**

3️⃣ **JSON là công cụ phổ biến để lưu và chia sẻ dữ liệu**

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

prompt 1 — Data-driven UI Architecture

Phân tích nguyên lý **UI = function(state)** trong Flutter.
Giải thích:
- vì sao UI phải được tạo từ dữ liệu
- cách state ảnh hưởng đến UI
- sự khác biệt giữa UI tĩnh và UI reactive

Liên hệ với cơ chế rebuild trong **Flutter**.

Prompt 2 — Persistent Data vs In-Memory Data

So sánh hai loại dữ liệu trong ứng dụng mobile:
- in-memory data
- persistent data

Phân tích:
- ưu điểm
- nhược điểm
- khi nào nên sử dụng mỗi loại.

Liên hệ với ứng dụng:
- Todo App
- Notes App
- Expense Tracker.

Prompt 3 — Source of Truth Architecture

Giải thích khái niệm **Single Source of Truth** trong quản lý dữ liệu.
Phân tích:
- vì sao nhiều nguồn dữ liệu dễ gây lỗi
- cách thiết kế một source of truth cho Todo App.

Prompt 4 — Reactive Programming Model

Phân tích cách Flutter áp dụng mô hình **reactive programming**.
Giải thích:
- state change
- widget rebuild
- UI update.

Liên hệ với:
- **StatefulWidget**
- **setState**

Prompt 5 — List Rendering Performance

Phân tích sự khác biệt giữa:
- **ListView**
- **ListView.builder**

Đánh giá:
- hiệu năng
- memory usage
- khả năng mở rộng khi danh sách có 1000+ items.

Prompt 6 — Data Flow trong Flutter Application

Phân tích luồng dữ liệu chuẩn trong ứng dụng Flutter.
Ví dụ:

PhucPT10@fpt.edu.vn

```
User Action
↓
Update Data
↓
State Change
↓
UI Rebuild
```

Giải thích vì sao thiết kế data flow rõ ràng giúp:

dễ debug
dễ maintain.

Prompt 7 — JSON Serialization Strategy

Phân tích vai trò của **JSON** trong ứng dụng mobile.
Giải thích:

serialization
deserialization
data interchange.

Liên hệ với thư viện:

dart:convert

Prompt 8 — Data Integrity trong Mobile App

Phân tích khái niệm **data integrity**.
Giải thích:

vì sao dữ liệu phải luôn nhất quán
những tình huống có thể gây lỗi dữ liệu.

Ví dụ:

crash app
sync lỗi
duplicate records.

Prompt 9 — Offline Data Strategy

Phân tích cách ứng dụng mobile hoạt động khi **không có internet**.
Giải thích:

local storage
cache
sync khi online.

Prompt 10 — Data Lifecycle trong Mobile App

Phân tích vòng đời dữ liệu trong một ứng dụng.
Ví dụ:

```
Create data
↓
Store data
↓
Read data
↓
Update data
↓
Backup data
```

Giải thích cách mỗi bước ảnh hưởng đến thiết kế ứng dụng

PhucPT10@fpt.edu.vn

Prompt 1 — Offline-first Architecture

Nghiên cứu kiến trúc **offline-first mobile app**.
Giải thích:
vì sao nhiều ứng dụng mobile phải hoạt động offline
cách thiết kế hệ thống sync.

Prompt 2 — Large List Performance Optimization

Phân tích cách tối ưu hiệu năng khi danh sách có:
10,000 items
scrolling liên tục.

Prompt 3 — Data Synchronization Strategy

Nghiên cứu các chiến lược đồng bộ dữ liệu giữa:
local storage
server API.
Ví dụ:
push sync
pull sync.

Prompt 4 — Conflict Resolution

Phân tích vấn đề **data conflict** khi nhiều thiết bị chỉnh sửa dữ liệu cùng lúc.
Ví dụ:
User chỉnh sửa note trên:
điện thoại
tablet.

Prompt 5 — JSON Data Schema Design

Nghiên cứu cách thiết kế JSON schema cho ứng dụng lớn.
Phân tích:
nested structure
versioning.

Prompt 6 — Data Migration

Phân tích vấn đề **data migration** khi app update.
Ví dụ:
Version 1:
```
Todo
title
```
Version 2:
```
Todo
title
priority
```

Prompt 7 — Backup Strategy

Nghiên cứu các chiến lược backup dữ liệu:
local backup
cloud backup.

Prompt 8 — Data Security

Phân tích cách bảo vệ dữ liệu người dùng.

PhucPT10@fpt.edu.vn

Ví dụ:
- encrypt JSON backup
- secure storage.

Prompt 9 — Error Handling trong Data Sync

Nghiên cứu cách xử lý lỗi khi:
- save data fail
- network error.

Prompt 10 — Scalable Data Model

Phân tích cách thiết kế data model có thể mở rộng.
Ví dụ:
Todo App → Task Management System.

Prompt 11 — Data Caching Strategy

Nghiên cứu cách caching dữ liệu để:
- tăng tốc UI
- giảm request server.

Prompt 12 — Event-driven Data Updates

Phân tích cách UI được cập nhật dựa trên events.
Ví dụ:
- WebSocket
- stream updates.

Prompt 13 — Multi-device Data Sync

Phân tích cách dữ liệu được đồng bộ giữa:
- phone
- tablet
- web.

Prompt 14 — Data Backup UX Design

Nghiên cứu cách thiết kế UX cho tính năng backup.
Ví dụ:
- export file
- auto backup.

Prompt 15 — Observability & Debugging Data

Phân tích cách developer debug dữ liệu trong production.
Ví dụ:
- log JSON
- analytics.

PhucPT10@fpt.edu.vn